

# Parallel Algorithms — GPU Programming Placement and Interview Guide

This guide covers eight reusable GPU algorithm patterns. Examples use CUDA-like pseudocode, but the ideas apply to CUDA, HIP, OpenCL, Metal, SYCL, and SIMD/SIMT machines.  $n$  denotes the number of input elements, a warp is a lockstep group of lanes, and “depth” means the length of the critical parallel path.

## Reduction

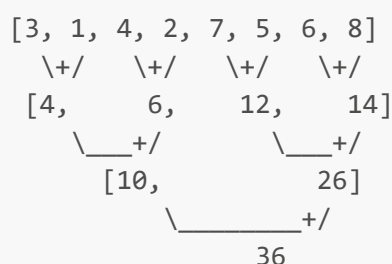
### 1. Overview

**Definition.** A reduction combines many values into one value using an operator: sum, product, minimum, maximum, logical AND/OR, or an aggregate such as  $(\text{sum}, \text{count})$ . For example, reducing  $[3, 1, 4, 2]$  with addition produces  $10$ .

Reduction matters because analytics, normalization, loss computation, convergence checks, database aggregation, and scientific simulations constantly summarize large arrays. Interviewers use it to test associativity, tree algorithms, synchronization, shared memory, warp primitives, numerical accuracy, and the difference between total work and parallel depth.

### 2. Core Idea

Imagine a knockout tournament. In each round, pairs combine; half the candidates remain. A sequential sum has  $n-1$  dependent additions. A balanced tree still performs  $n-1$  additions— $O(n)$  work—but has only  $O(\log n)$  rounds.



On a GPU, each block usually reduces a contiguous tile into one partial result. A later kernel reduces the partials. Within a block, threads load coalesced values, combine them in shared memory or with warp shuffles, and synchronize between rounds.

```

__global__ void block_sum(const float* x, float* partial, size_t n) {
    extern __shared__ float s[];
    unsigned t = threadIdx.x;
    size_t i = 2ull * blockIdx.x * blockDim.x + t;
    float v = i < n ? x[i] : 0.0f;
  
```

```

    if (i + blockDim.x < n) v += x[i + blockDim.x];
    s[t] = v;
    __syncthreads();

    for (unsigned stride = blockDim.x / 2; stride > 0; stride >= 1) {
        if (t < stride) s[t] += s[t + stride];
        __syncthreads();
    }
    if (t == 0) partial[blockIdx.x] = s[0];
}

```

The identity `0` safely represents missing elements for sum. Host code repeatedly launches the kernel until one partial remains, or uses a final CPU reduction when only a few values remain.

### 3. Important Subtopics

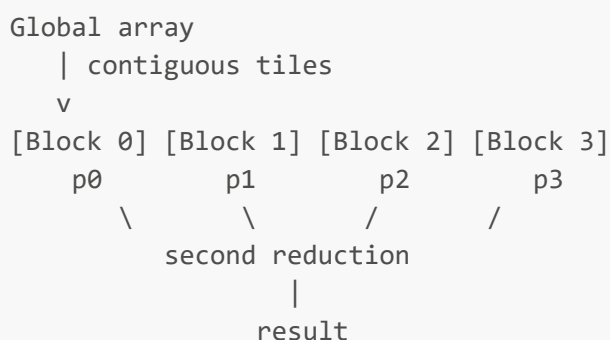
| Subtopic             | Meaning and why it matters                                                                                                                   | Example                                                                      | Common interview angle                                               |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Associativity        | Regrouping must preserve the mathematical result: $(a \text{ op } b) \text{ op } c = a \text{ op } (b \text{ op } c)$ . It permits a tree.   | <code>min</code> is associative.                                             | Addition of floating-point values is only approximately associative. |
| Identity element     | A neutral value handles inactive lanes and padding.                                                                                          | <code>0</code> for sum, <code>1</code> for product, <code>+∞</code> for min. | Choose the correct identity, especially for negative maxima.         |
| Block reduction      | One block reduces a tile and emits one partial.                                                                                              | 256 threads load up to 512 items.                                            | Explain why blocks cannot normally synchronize inside one kernel.    |
| Multi-pass reduction | Partial results are recursively reduced.                                                                                                     | $10^6 \rightarrow 1954 \rightarrow 4 \rightarrow 1$ .                        | Count launches and temporary storage.                                |
| Shared-memory tree   | Threads exchange partials through fast on-chip storage.                                                                                      | Halve active threads each round.                                             | Barriers are required between producer/consumer rounds.              |
| Warp shuffle         | Lanes exchange register values without shared memory.                                                                                        | <code>v += shfl_down(v, 16);</code>                                          | Active mask and warp-width assumptions matter.                       |
| Divergence           | Interleaved addressing makes predicates such as <code>tid % (2s) == 0</code> inefficient. Sequential addressing keeps active lanes together. | First half adds second half.                                                 | Compare naive and optimized trees.                                   |

| Subtopic            | Meaning and why it matters                                                                                                                   | Example                          | Common interview angle                                        |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|---------------------------------------------------------------|
| Bank conflicts      | Shared-memory addresses may map to the same bank and serialize.                                                                              | Poor strided layouts.            | Modern sequential reductions usually avoid serious conflicts. |
| Atomics             | Many threads update one location atomically. Simple but contended and order-dependent.                                                       | <code>atomicAdd(total,v).</code> | Atomics can be appropriate for few blocks or newer hardware.  |
| Numerical stability | Tree order changes rounding. Pairwise summation is often more accurate than left-to-right, but not deterministic across all implementations. | Kahan per thread plus tree.      | Accuracy, determinism, and throughput trade off.              |

## 4. Real-World Example

During neural-network training, a loss kernel produces one loss per sample. A reduction sums those losses and divides by the batch size. A production implementation keeps data on the GPU, fuses local accumulation into the producer when possible, accumulates FP16/BF16 inputs in FP32, and uses collective all-reduce when the batch spans multiple GPUs.

## 5. Diagrams / Mental Models



| Measure              | Sequential fold | Parallel tree         |
|----------------------|-----------------|-----------------------|
| Work                 | $O(n)$          | $O(n)$                |
| Depth                | $O(n)$          | $O(\log n)$           |
| Extra global storage | none            | one partial per block |
| Reordering           | no              | yes                   |

## 6. Common Interview Questions

| Question                                        | Clear answer                                                                                                    | Key points expected                                   | Common mistake                                            |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------|
| 1. What is a reduction?                         | Combining $n$ items into one aggregate with an associative operator.                                            | Examples and operator requirements.                   | Calling any loop a reduction.                             |
| 2. Why must the operator be associative?        | Parallel workers group operands differently; associativity makes groupings equivalent.                          | Tree reordering.                                      | Requiring commutativity instead; order can remain fixed.  |
| 3. What are work and depth?                     | $O(n)$ total combines and $O(\log n)$ dependent rounds.                                                         | Work-depth model.                                     | Claiming total work is $O(\log n)$ .                      |
| 4. Why use shared memory?                       | It lets threads in a block exchange partials with lower latency and traffic than global memory.                 | Reuse plus synchronization.                           | Saying shared memory needs no barriers.                   |
| 5. Why multiple kernels?                        | Ordinary blocks have no grid-wide barrier; a launch boundary safely separates levels.                           | GPU execution model.                                  | Assuming blocks run simultaneously.                       |
| 6. What does <code>__syncthreads()</code> do?   | It waits for all threads in a block and orders their shared-memory accesses.                                    | Block scope; all participating threads must reach it. | Treating it as a device-wide barrier.                     |
| 7. How do non-power-of-two sizes work?          | Bounds-check loads and use the identity for absent items; a general tree/library handles block shape.           | Neutral padding.                                      | Reading past the input.                                   |
| 8. Why load two elements per thread?            | It performs useful combining during coalesced loads and halves the number of active partials/blocks.            | Less overhead, same asymptotic work.                  | Assuming two is universally optimal.                      |
| 9. Are floating-point reductions deterministic? | Not necessarily; different legal trees and atomic orders round differently.                                     | Non-associativity and reproducibility.                | Calling it a race when synchronization is correct.        |
| 10. When are atomics acceptable?                | When contention is low, input is small, hardware supports the type well, or simplicity beats a multi-pass tree. | Measure rather than categorically reject.             | One atomic per element on a hot address without analysis. |

## 7. Deep-Dive Questions

1. **Can a non-commutative operator be reduced?** Yes if it is associative and the algorithm preserves operand order. Matrix multiplication and string concatenation are examples; arbitrary atomic arrival

order would be invalid.

2. **How would you reduce (value, index) for argmax?** Define an associative comparator over pairs and a deterministic tie-breaker, such as smaller index. Use  $(-\infty, \text{invalid\_index})$  as identity.
3. **How do cooperative groups change the design?** A cooperative launch can provide a grid-wide synchronization point, enabling a single-kernel reduction, but it imposes residency and launch constraints; multi-pass reduction is more portable.
4. **How can accuracy improve?** Accumulate low-precision inputs in a wider type, use pairwise trees, Kahan/Neumaier compensation in per-thread chunks, or reproducible algorithms. Each adds cost.
5. **How is multi-GPU reduction performed?** Each GPU computes local partials, then a collective such as ring/tree all-reduce combines and distributes results. Communication bandwidth and topology become central.

## 8. Comparison Tables

| Strategy               | Global traffic           | Synchronization           | Strength                 | Weakness                   |
|------------------------|--------------------------|---------------------------|--------------------------|----------------------------|
| One atomic per element | high contention          | atomic serialization      | Tiny implementation      | Poor for a hot destination |
| Atomic per block       | low                      | block barriers + atomics  | Often practical          | Result order may vary      |
| Multi-pass tree        | linear, then shrinking   | kernel boundaries         | Scalable and predictable | Extra launches/storage     |
| Warp-shuffle hybrid    | low                      | warp sync + block barrier | Fast final stages        | More hardware-specific     |
| Library primitive      | implementation-dependent | handled internally        | Usually best default     | Less educational/control   |

## 9. Common Mistakes

- Confusing  $O(\log n)$  depth with  $O(\log n)$  total work.
- Reducing with a non-associative operation such as subtraction.
- Putting `__syncthreads()` inside a branch that not all block threads take.
- Choosing 0 as the identity for maximum when all inputs can be negative.
- Assuming volatile memory or warp lockstep replaces correct synchronization.
- Ignoring overflow, NaNs, empty input, and floating-point reproducibility.
- Optimizing the last warp while global memory traffic or launch overhead dominates.

## 10. Edge Cases / Special Cases

- **Empty input:** return a defined identity or report that no aggregate exists; argmin/argmax needs an explicit validity policy.
- **NaN:** decide whether min/max propagates NaN or ignores it; library and language semantics vary.
- **Integer overflow:** use a wider accumulator or documented modular/saturating behavior.
- **Very small input:** CPU execution or a single block may be faster.
- **Huge input:** use grid-stride per-thread accumulation and 64-bit indexing.

- **Non-power-of-two block:** either use a general algorithm or carefully fold the excess; a simple halving loop can drop values.
- **Signed zero/infinity:** floating-point min/max semantics can affect reproducibility.

## 11. How to Explain in Interview

"A reduction converts many elements to one aggregate using an associative operator. On a GPU I let each block reduce a coalesced tile in shared memory or registers, write one partial, and recursively reduce the partials because ordinary blocks lack a global barrier. It has  $O(n)$  work and  $O(\log n)$  depth; I also handle identities, bounds, synchronization, and floating-point ordering."

## 12. Quick Revision Notes

- Definition: many-to-one associative combine.
- Complexity:  $O(n)$  work,  $O(\log n)$  tree depth.
- Identity handles missing elements; associativity enables regrouping.
- Block-local barrier is not grid-wide synchronization.
- Tree/multi-pass, warp shuffle, and hierarchical atomics are common designs.
- Trap: floating-point addition is not exactly associative.

## 13. Practice Tasks

1. Implement CPU sequential and tree reductions; compare results for adversarial floats.
2. Write a CUDA block sum with bounds-safe two-elements-per-thread loading.
3. Extend it to min, max, and argmax with deterministic ties.
4. Benchmark block sizes and atomic-per-block versus multi-pass versions.
5. Use a profiler to inspect bandwidth, occupancy, and barrier stalls.
6. Explain why subtraction cannot directly replace addition in the tree.

## 14. Final Cheat Sheet

| Item            | Must remember                                                                        |
|-----------------|--------------------------------------------------------------------------------------|
| Core definition | Many values $\rightarrow$ one value using an associative operator                    |
| Main pattern    | Per-thread chunk $\rightarrow$ warp $\rightarrow$ block $\rightarrow$ grid/multi-GPU |
| Complexity      | $O(n)$ work, $O(\log n)$ depth                                                       |
| Most asked      | Associativity, barriers, shared memory, atomics, FP accuracy                         |
| Key comparison  | Multi-pass scales; atomics are simpler but may contend                               |
| One-line answer | "Reduction is a hierarchical associative combine from $n$ inputs to one output."     |

## Prefix Sum

### 1. Overview

**Definition.** Prefix sum, or scan, transforms an array into cumulative results. Inclusive sum scan of `[3,1,4,2]` is `[3,4,8,10]`; exclusive scan is `[0,3,4,8]`.

Scan matters because it turns local flags or sizes into global positions. GPU compaction, radix sort, stream allocation, token offsets, sparse matrix construction, and graph frontiers all use it. Interviewers ask it because scan exposes dependency removal, work-efficient versus step-efficient trees, synchronization, hierarchical composition, and inclusive/exclusive semantics.

## 2. Core Idea

A sequential scan seems inherently dependent: output `i` uses output `i-1`. Associativity lets a parallel tree first compute group totals and then distribute the prefix accumulated before each group.

For exclusive scan of `[3,1,4,2]`:

Up-sweep (build sums):  
10  
 / \  
4 6  
/\ /\  
3 1 4 2

Down-sweep (distribute prefixes):  
[0,3,4,8]  
root starts 0; left inherits prefix,  
right gets prefix + left-subtree sum

The Blelloch algorithm has two phases. Up-sweep constructs a reduction tree. Set the root to the identity. Down-sweep swaps and combines partials to produce an exclusive scan. It uses  $O(n)$  work and  $O(\log n)$  depth. An inclusive result follows from `inclusive[i] = exclusive[i] + input[i]` for addition.

For arrays larger than a block:

- 1. Each block scans its tile and writes its block total.
- 2. Scan the array of block totals.
- 3. Add each block’s scanned offset to every result in that block.

## 3. Important Subtopics

| Subtopic       | Meaning and why it matters                                                                  | Example                          | Common interview angle                   |
|----------------|---------------------------------------------------------------------------------------------|----------------------------------|------------------------------------------|
| Inclusive scan | Output <code>i</code> includes input <code>i</code> .                                       | <code>[2,5,1] → [2,7,8]</code> . | Convert from exclusive form.             |
| Exclusive scan | Output <code>i</code> contains all values before <code>i</code> ; first output is identity. | <code>[2,5,1] → [0,2,7]</code> . | Ideal for zero-based offsets.            |
| Hillis–Steele  | Each round adds an increasingly distant predecessor. $O(n \log n)$ work, $O(\log n)$ depth. | Offsets 1,2,4,...                | Simple but work-inefficient.             |
| Blelloch scan  | Up-sweep plus down-sweep. $O(n)$ work, $O(\log n)$ depth.                                   | Tree over a tile.                | Explain root reset and exclusive result. |

| Subtopic            | Meaning and why it matters                                                                     | Example                            | Common interview angle                           |
|---------------------|------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------|
| Hierarchical scan   | Block scans + scan block totals + uniform add.                                                 | Large arrays.                      | How block boundaries receive correct offsets.    |
| Segmented scan      | Prefixes restart at marked segment heads.                                                      | Values grouped by key.             | Carry both value and head flag.                  |
| Warp scan           | Shuffles exchange values among lanes without shared memory.                                    | <code>shfl_up</code> at 1,2,4,...  | Guard lanes whose source is invalid.             |
| Operator/identity   | Any associative operator works, not only sum.                                                  | Prefix maximum.                    | State the algebraic requirement.                 |
| Decoupled look-back | Blocks publish partial status and obtain earlier prefixes without a separate global scan pass. | High-performance device-wide scan. | Advanced single-pass design and memory ordering. |

## 4. Real-World Example

A log-processing GPU marks each valid record with `1` and invalid record with `0`. An exclusive scan of the flags gives every valid record a unique compacted destination. If flags are `[1,0,1,1]`, offsets are `[0,1,1,2]`; valid threads write to positions `0,1,2`. The final output size is `offset[n-1] + flag[n-1]`.

## 5. Diagrams / Mental Models

```
items:  [A B C D E]
keep?:  [1 0 1 1 0]
scan:   [0 1 1 2 3]   exclusive
scatter: A---->0
          C---->1
          D---->2
result:  [A C D]
```

| Algorithm     | Work                            | Depth       | Typical role              |
|---------------|---------------------------------|-------------|---------------------------|
| Sequential    | $O(n)$                          | $O(n)$      | CPU/small input           |
| Hillis–Steele | $O(n \log n)$                   | $O(\log n)$ | Simple warp/small scan    |
| Blelloch      | $O(n)$                          | $O(\log n)$ | Work-efficient block scan |
| Hierarchical  | $O(n)$ $O(\log n)$ conceptually |             | Device-wide scan          |

## 6. Common Interview Questions

| Question                                  | Clear answer                                                                                | Key points expected                 | Common mistake                                      |
|-------------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------|
| 1. What is prefix sum?                    | It emits the aggregate of every prefix of an input sequence.                                | Inclusive/exclusive examples.       | Describing only total sum.                          |
| 2. Inclusive vs exclusive?                | Inclusive contains the current element; exclusive starts with identity and excludes it.     | Correct sample output.              | Off-by-one explanation.                             |
| 3. Why is scan useful?                    | It converts counts/flags into deterministic offsets.                                        | Compaction, radix sort, allocation. | Saying it is only cumulative sum.                   |
| 4. Why can it be parallel?                | Associativity allows a tree to summarize groups and propagate preceding totals.             | Dependency restructuring.           | Claiming each output is independent.                |
| 5. Hillis–Steele complexity?              | $O(n \log n)$ work and $O(\log n)$ depth.                                                   | Every element works each round.     | Calling it work-efficient.                          |
| 6. Blelloch complexity?                   | $O(n)$ work and $O(\log n)$ depth.                                                          | Up-sweep/down-sweep.                | Forgetting both phases.                             |
| 7. How scan multiple blocks?              | Scan tiles, scan tile totals, uniformly add offsets.                                        | Three-stage hierarchy.              | Adding the unscanned block sum.                     |
| 8. What is segmented scan?                | A scan whose state resets at segment-start flags.                                           | Irregular grouped data.             | Launching one kernel per segment as the definition. |
| 9. How get output count after compaction? | <code>exclusive[n-1] + flag[n-1]</code> , or retain the total from the scan.                | Last-element edge case.             | Using only <code>scan[n-1]</code> .                 |
| 10. Can scan use another operator?        | Yes, any associative operator with an identity; order must be preserved if non-commutative. | Generalized scan.                   | Requiring addition/commutativity.                   |

## 7. Deep-Dive Questions

- Why is Blelloch work-efficient?** At tree level  $d$ , about  $n/2^{(d+1)}$  nodes combine. The geometric sum across each phase is  $O(n)$ , unlike Hillis–Steele’s roughly  $n$  operations per level.
- How do you make a segmented operator associative?** Combine pairs  $(\text{head}, \text{value})$  so a right-hand head discards the left aggregate; otherwise combine values and propagate whether a head occurred. A correct associative pair operator enables the same scan machinery.
- Can scan be in place?** Block scan can be in place with careful barriers because each round preserves needed values. Device-wide algorithms require controlled intermediate state; library contracts determine permitted aliasing.

4. **Why might a theoretically work-inefficient warp scan be fast?** A warp is small and shuffle steps are cheap; avoiding shared-memory traffic and barriers can outweigh a few extra operations.
5. **What does decoupled look-back optimize?** It overlaps local scan with propagation of block prefixes, reducing global passes and memory traffic. Correct publication states and acquire/release-style ordering prevent observing incomplete partials.

## 8. Comparison Tables

| Aspect             | Inclusive                                 | Exclusive                                   |
|--------------------|-------------------------------------------|---------------------------------------------|
| First output       | $x[0]$                                    | identity                                    |
| Output $i$         | $x[0] \text{ op } \dots \text{ op } x[i]$ | $x[0] \text{ op } \dots \text{ op } x[i-1]$ |
| Natural use        | Running totals                            | Starting offsets                            |
| Conversion for sum | $\text{inc}[i] = \text{exc}[i] + x[i]$    | shift inclusive right; insert 0             |

| Aspect      | Reduction       | Scan                   |
|-------------|-----------------|------------------------|
| Outputs     | one             | one per input          |
| Information | total aggregate | every prefix aggregate |
| Tree        | mostly up-sweep | up-sweep + down-sweep  |
| Typical use | statistics      | offsets/compaction     |

## 9. Common Mistakes

- Mixing inclusive and exclusive semantics in code or output-size formulas.
- Calling Hillis–Steele  $O(n)$  work because it has  $O(\log n)$  rounds.
- Omitting a barrier between shared-memory read/write rounds.
- Scanning each block but never adding prefixes of earlier blocks.
- Using scan with a non-associative operator.
- Assuming all inputs or block sizes are powers of two.
- Forgetting that overflow changes offset correctness and can cause out-of-bounds writes.

## 10. Edge Cases / Special Cases

- **Empty input:** output is empty; the total is the identity.
- **One item:** inclusive output is the item; exclusive output is the identity.
- **Non-power-of-two length:** pad logically with identity or use a general implementation.
- **Large totals:** offsets often need 64-bit integers even when flags are 32-bit.
- **Segment at index 0:** normally required or treated as implicit.
- **Floating point:** reordering changes rounding, as with reduction.
- **Aliasing:** an output overwriting input too early can corrupt later rounds.

## 11. How to Explain in Interview

“Prefix sum produces every cumulative prefix, either inclusive or exclusive. A work-efficient GPU scan builds a sum tree in an up-sweep and propagates offsets in a down-sweep, giving  $O(n)$  work and  $O(\log n)$  depth. For

multiple blocks, I scan tiles, scan block totals, then add each block's prefix. It is the standard way to turn flags or sizes into output positions."

## 12. Quick Revision Notes

- Inclusive includes current item; exclusive begins with identity.
- Blelloch: up-sweep, root reset, down-sweep.
- Large scan: local scan → scan block totals → uniform add.
- Scan powers compaction, radix partitioning, sparse offsets, and allocation.
- $O(n)$  work,  $O(\log n)$  depth for work-efficient scan.
- Trap: last exclusive value is not necessarily the total.

## 13. Practice Tasks

1. Trace inclusive and exclusive scans by hand for seven elements.
2. Implement Hillis–Steele and count operations for  $n=8, 16, 32$ .
3. Implement a Blelloch block scan with logical identity padding.
4. Build stream compaction from predicate → scan → scatter.
5. Extend scan to prefix maximum and segmented sum.
6. Test empty, one-element, non-power-of-two, and 64-bit-offset cases.

## 14. Final Cheat Sheet

| Item                     | Must remember                                                                |
|--------------------------|------------------------------------------------------------------------------|
| Core definition          | Array → aggregate of every prefix                                            |
| Main variants            | Inclusive, exclusive, segmented                                              |
| Best-known basic pattern | Work-efficient up-sweep/down-sweep                                           |
| Complexity               | $O(n)$ work, $O(\log n)$ depth                                               |
| Most asked               | Scan vs reduction, hierarchy, compaction, off-by-one                         |
| One-line answer          | "Scan parallelizes cumulative dependencies and turns counts into positions." |

## Parallel Sorting

### 1. Overview

**Definition.** Parallel sorting rearranges keys—often key/value records—into nondecreasing order using many workers. GPUs favor algorithms with regular control flow and memory access, especially radix sort, merge sort, sorting networks, and sample sort.

Sorting matters in databases, graphics, particle simulation, search indexing, grouping, deduplication, and sparse computation. Interviewers ask it to test algorithm selection, stability, work/depth, data movement, scan-based partitioning, divergence, and the difference between comparison and non-comparison sorting.

### 2. Core Idea

Parallel sort replaces one long chain of comparisons with independent local work plus structured merging or partitioning. In merge sort, workers sort tiles and merge adjacent runs in parallel. In radix sort, each pass groups keys by a digit: classify digits, histogram counts, scan counts into bucket offsets, then scatter stably.

Example: one radix bit over  $[6, 3, 4, 1]$  ( $110, 011, 100, 001$ ) produces zeros  $[6, 4]$ , then ones  $[3, 1]$ . Repeating stable passes from least-significant digit to most-significant digit yields sorted order.

keys  $\rightarrow$  digit extraction  $\rightarrow$  histogram  $\rightarrow$  scan bucket counts  $\rightarrow$  stable scatter  
repeat for each digit

### 3. Important Subtopics

| Subtopic             | Meaning / importance                                                        | Example                                                                          | Interview angle                                            |
|----------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------|
| Radix sort           | Sorts fixed-width keys by digits without key comparisons.                   | 4 or 8 bits/pass.                                                                | $O(kn)$ for $k$ digit passes; uses histogram/scan/scatter. |
| Parallel merge sort  | Sort tiles, then merge runs of doubling size.                               | Merge path divides two sorted arrays.                                            | $O(n \log n)$ work and balanced partitioning.              |
| Bitonic/network sort | Fixed compare-exchange network, regular and branch-light.                   | Fast small block-resident tiles.                                                 | $O(n \log^2 n)$ work; power-of-two padding.                |
| Sample sort          | Samples splitters, partitions into buckets, sorts buckets.                  | Distributed/GPU large data.                                                      | Skew and load balance.                                     |
| Stability            | Equal keys retain original order.                                           | Required by LSD radix passes.                                                    | Stable key/value sorting and multi-key sort.               |
| Key/value sort       | Values move with their keys.                                                | Sort $(user\_id, event)$ .                                                       | Bandwidth cost and indirect payloads.                      |
| Local vs global      | Shared-memory tile sorts reduce global traffic; global phases combine them. | Block radix sort then device scatter.                                            | Hierarchical design.                                       |
| Signed/float keys    | Bit representations require monotonic transforms.                           | Flip sign bit for unsigned ordering; float transform treats negatives carefully. | NaNs and signed zero policy.                               |

### 4. Real-World Example

A GPU database sorts  $(customer\_id, transaction)$  records. Equal IDs become contiguous, enabling a segmented reduction for totals per customer. A radix sort is attractive because IDs are fixed-width integers; values are moved alongside keys, and stability preserves timestamp order when a prior pass sorted by time.

## 5. Diagrams / Mental Models

```

LSD radix pass: [key,value]
  | digit d
  v
per-block counts → global bucket offsets (scan)
  | stable local rank + bucket offset
  v
scatter to alternate buffer → next digit

```

| Algorithm | Work                   | Best fit                      | Main risk            |
|-----------|------------------------|-------------------------------|----------------------|
| Radix     | $O(kn)$                | fixed-width integer-like keys | extra buffers/passes |
| Merge     | $O(n \log n)$          | general comparator            | merge traffic        |
| Bitonic   | $O(n \log^2 n)$        | small fixed tiles             | extra comparisons    |
| Sample    | expected $O(n \log n)$ | large/distributed sets        | skewed buckets       |

## 6. Common Interview Questions

| Question                                | Answer                                                                                                         | Expected                            | Common mistake                                     |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------|----------------------------------------------------|
| 1. Why not GPU quicksort by default?    | Irregular recursion, divergence, and unbalanced partitions are awkward.                                        | GPU regularity/load balance.        | Claiming quicksort cannot be parallel.             |
| 2. Why is radix sort fast?              | Fixed passes use regular classify/count/scan/scatter operations and avoid comparisons.                         | Key width and bandwidth.            | Calling it universally $O(n)$ without digit count. |
| 3. Why must LSD radix passes be stable? | Later digit passes must preserve ordering established by lower digits.                                         | Stability reasoning.                | Confusing LSD and MSD behavior.                    |
| 4. What is a sorting network?           | A data-independent sequence of compare-exchanges.                                                              | Regular execution; bitonic example. | Saying it has optimal work.                        |
| 5. How parallelize merge?               | Partition output diagonals via binary search/merge path, then merge independent ranges.                        | Balanced output ownership.          | One thread per entire merge.                       |
| 6. Comparison-sort lower bound?         | Sequential comparison sorting needs $\Omega(n \log n)$ comparisons; radix escapes by using key representation. | Model qualification.                | Applying lower bound to radix.                     |
| 7. In-place or out-of-place?            | Many GPU radix/merge implementations ping-pong buffers for coalesced                                           | Memory/traffic tradeoff.            | Assuming in-place is always                        |

| Question                         | Answer                                                                                       | Expected                | Common mistake                        |
|----------------------------------|----------------------------------------------------------------------------------------------|-------------------------|---------------------------------------|
|                                  | deterministic writes.                                                                        |                         | faster.                               |
| 8. How handle duplicates?        | Correct bucket/merge logic; choose stable behavior and deterministic tie handling if needed. | Equality policy.        | Dropping duplicates during partition. |
| 9. When is CPU faster?           | Small arrays, transfer-dominated workloads, complex comparators, or data already on CPU.     | End-to-end cost.        | Comparing only throughput.            |
| 10. What determines radix width? | More bits mean fewer passes but more buckets, shared memory, and scatter complexity.         | Tune resource tradeoff. | Always maximizing bits/pass.          |

## 7. Deep-Dive Questions

1. **How does merge path work?** Each output rank corresponds to a diagonal through the two input arrays. Binary search finds where that diagonal intersects the merge boundary, giving independent balanced ranges.
2. **How sort floating-point keys?** Transform IEEE bits to a monotonic unsigned ordering, define placement of NaNs and signed zeros, sort transformed bits, and preserve original payloads.
3. **How does radix sort compute stable positions?** Position equals global bucket base plus counts from earlier blocks plus the item's stable rank among same-digit items in its block.
4. **What if sample-sort buckets are skewed?** Oversample, choose better splitters, recursively repartition large buckets, and use size-aware scheduling.
5. **How sort records with large payloads?** Sort compact (key, index) pairs, then gather payloads once; moving large records every pass wastes bandwidth.

## 8. Comparison Tables

| Aspect           | Radix                     | Merge                 | Bitonic               |
|------------------|---------------------------|-----------------------|-----------------------|
| Key restriction  | digit-extractable         | comparator            | comparator            |
| Stability        | natural if scatter stable | natural with tie rule | must be designed      |
| Access/control   | regular                   | moderately regular    | highly regular        |
| Small-tile use   | good                      | good                  | excellent             |
| Large global use | excellent                 | good                  | usually too much work |

## 9. Common Mistakes

- Choosing an algorithm only by asymptotic complexity and ignoring traffic/divergence.
- Forgetting stability in LSD radix sort or key/value association.
- Treating signed integers/floats as ordinary unsigned bits.
- Assuming an in-place algorithm necessarily uses less traffic.

- Ignoring duplicates, NaNs, already-sorted input, and skew.
- Comparing GPU kernel time against CPU end-to-end time.

## 10. Edge Cases / Special Cases

Empty and one-item inputs need no work. Non-power-of-two network inputs require sentinels or guarded compares. All-equal keys stress stability but should not cause imbalance. Radix passes may skip digits whose bits are identical. Comparator consistency is mandatory. For floats, explicitly define NaN, infinity, and  $-0/+0$ . Sizes and offsets may need 64 bits.

## 11. How to Explain in Interview

“GPU sorting favors regular, bandwidth-efficient algorithms. For fixed-width keys I would usually choose radix sort: each stable digit pass performs classify, histogram, scan, and scatter. For arbitrary comparators I would use parallel merge sort; bitonic sort is useful for small shared-memory tiles. Selection depends on key type, stability, size, payload, and transfer cost.”

## 12. Quick Revision Notes

- Radix = digits; merge = comparator; bitonic = fixed network.
- LSD radix requires stable passes.
- Sorting is often bandwidth-bound due to repeated global reads/writes.
- Move indices instead of large payloads.
- Trap: comparison lower bound does not cover radix sorting.

## 13. Practice Tasks

1. Trace a stable 2-bit radix pass and compute each item's output index.
2. Implement bitonic sort for one power-of-two block, then add padding.
3. Merge two arrays by assigning output partitions with binary search.
4. Benchmark CPU/GPU crossover and key-only versus key/value sorting.
5. Test duplicates, reverse order, all equal, signed extremes, and NaNs.

## 14. Final Cheat Sheet

| Item               | Must remember                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------|
| Core               | Many workers order keys through regular partition/merge stages                                       |
| Default fixed key  | Stable radix sort                                                                                    |
| General comparator | Parallel merge sort                                                                                  |
| Small tile         | Bitonic/network sort                                                                                 |
| Most asked         | Stability, radix pipeline, complexity, traffic, skew                                                 |
| One-line answer    | “Parallel sorting trades sequential decisions for regular classify, scan, scatter, or merge phases.” |

# Histogram

## 1. Overview

**Definition.** A histogram counts how many inputs fall into each category or numeric bin. For bins `0..3`, input `[2,0,2,3,2,0]` yields `[2,0,3,1]`.

Histograms power image processing, analytics, probability distributions, radix sort, feature extraction, and database grouping. Interviewers ask because the input reads are parallel but output updates collide, making it a compact test of atomics, privatization, contention, memory hierarchy, and skew.

## 2. Core Idea

Giving every item a thread is easy; safely updating shared counters is hard. A naive `bins[x[i]]++` races. `atomicAdd` makes it correct but many threads may serialize on popular bins. The standard GPU pattern privatizes histograms per thread/warp/block, then reduces private counters into the global histogram.

```
// Conceptual block-private histogram
initialize shared_bins to 0 cooperatively;
barrier;
for (i = global_tid; i < n; i += total_threads)
    atomicAdd(&shared_bins[bin(x[i])], 1);
barrier;
for (b = threadIdx.x; b < B; b += blockDim.x)
    atomicAdd(&global_bins[b], shared_bins[b]);
```

This changes up to `n` contended global atomics into roughly `blocks × B` global atomics, while faster shared-memory atomics absorb local collisions.

## 3. Important Subtopics

| Subtopic             | Meaning / importance                                                 | Example                                  | Interview angle                         |
|----------------------|----------------------------------------------------------------------|------------------------------------------|-----------------------------------------|
| Bin mapping          | Converts an item to a valid category.                                | <code>floor((x-low)/width)</code> .      | Boundary and out-of-range policy.       |
| Atomic update        | Indivisible read-modify-write prevents lost counts.                  | <code>atomicAdd(&amp;bins[b],1)</code> . | Correctness does not imply scalability. |
| Privatization        | Keep per-thread/warp/block counters, then merge.                     | One shared histogram per block.          | Memory versus contention tradeoff.      |
| Skew                 | Popular bins create hot spots.                                       | Most pixels near black.                  | Worst case all items in one bin.        |
| Shared-memory limits | <code>B × counter_size</code> must fit; replication increases usage. | 256 bins × 4 bytes.                      | Occupancy tradeoff.                     |

| Subtopic           | Meaning / importance               | Example                           | Interview angle                        |
|--------------------|------------------------------------|-----------------------------------|----------------------------------------|
| Counter width      | Counts can overflow.               | Use 64-bit for billions of items. | Atomic support/performance.            |
| Weighted histogram | Add a weight rather than 1.        | Probability mass per class.       | Floating-point nondeterminism.         |
| Joint histogram    | Counts combinations of categories. | 2D image intensity pairs.         | Bin explosion and sparse alternatives. |

## 4. Real-World Example

An image pipeline computes a 256-bin luminance histogram for exposure control. Each block processes a coalesced pixel tile into shared counters, then merges only 256 block totals globally. A following scan forms a cumulative distribution function used for histogram equalization.

## 5. Diagrams / Mental Models

```

items → [block-private bins]
items → [block-private bins]
items → [block-private bins]
           |
           | → merge/reduce → global bins
           |
           |
local contention                    few global updates

```

| Design            | Memory                             | Contention         | Best case                       |
|-------------------|------------------------------------|--------------------|---------------------------------|
| Global atomics    | B                                  | high               | small/uniform workload          |
| Per-block bins    | $\text{blocks} \times B$ logically | medium             | bins fit shared memory          |
| Per-warp replicas | $\text{warps} \times B$ per block  | lower              | heavy local collisions          |
| Sort + run count  | large temporary                    | avoids hot atomics | extreme skew/other sorting need |

## 6. Common Interview Questions

| Question                           | Answer                                                                  | Expected                               | Common mistake                       |
|------------------------------------|-------------------------------------------------------------------------|----------------------------------------|--------------------------------------|
| 1. Why does naive increment race?  | Multiple threads can read the same old count and overwrite one another. | Read-modify-write race.                | Assuming writes "eventually add up." |
| 2. What does atomic add guarantee? | Each update is indivisible for that address.                            | Correct count, possible serialization. | Calling all atomics globally serial. |
| 3. Why privatize?                  | It distributes collisions and reduces expensive global updates.         | Hierarchical aggregation.              | Forgetting the merge phase.          |

| Question                                  | Answer                                                                            | Expected               | Common mistake                                            |
|-------------------------------------------|-----------------------------------------------------------------------------------|------------------------|-----------------------------------------------------------|
| 4. What if bins do not fit shared memory? | Tile bins across passes, use sparse/local caches, global atomics, or sort/reduce. | Resource-aware choice. | Allocating unlimited shared memory.                       |
| 5. Effect of skew?                        | Hot bins serialize atomics and dominate runtime.                                  | Distribution matters.  | Benchmarking only uniform data.                           |
| 6. How initialize bins?                   | Threads cooperatively zero them, then barrier before updates.                     | Synchronization.       | Letting every thread clear all bins.                      |
| 7. How handle values at upper bound?      | Define half-open ranges and explicitly place/clamp the maximum.                   | Boundary policy.       | Producing bin <b>B</b> .                                  |
| 8. Why use 64-bit counters?               | Maximum count may exceed $2^{32}-1$ .                                             | Overflow analysis.     | Choosing width from bin count.                            |
| 9. Histogram vs reduction?                | Histogram is a many-to-few keyed reduction; each key chooses a destination bin.   | Collision pattern.     | Treating bins as independent input regions automatically. |
| 10. When sort first?                      | If atomics are pathological, bins are sparse/huge, or sorted data is also needed. | Cost tradeoff.         | Sorting unconditionally.                                  |

## 7. Deep-Dive Questions

1. **How can warp aggregation reduce atomics?** Lanes with the same bin form a group using match/ballot primitives; one leader performs an atomic add of the group size.
2. **Why replicate shared bins?** Multiple sub-histograms map the same logical bin to different physical counters, spreading bank/atomic pressure; a local reduction merges replicas.
3. **How build a sparse histogram?** Store only observed (**key, count**) entries via sort-and-reduce or a bounded shared hash table, then merge globally.
4. **Are weighted float histograms deterministic?** Usually not when atomic arrival order changes; integer counts are exact unless overflow occurs. Deterministic float results need a prescribed order or reproducible accumulator.
5. **How does histogram feed radix sort?** Per-digit counts are scanned into bucket starting offsets; each key combines the bucket base with a stable local rank.

## 8. Comparison Tables

| Aspect      | Histogram        | Reduction               | Scan                   |
|-------------|------------------|-------------------------|------------------------|
| Outputs     | <b>B</b> bins    | one aggregate           | <b>n</b> prefixes      |
| Destination | data-dependent   | one                     | index-aligned          |
| Main hazard | write collisions | combine synchronization | dependency propagation |

| Aspect              | Histogram     | Reduction      | Scan          |
|---------------------|---------------|----------------|---------------|
| Common optimization | privatization | tree hierarchy | up/down sweep |

## 9. Common Mistakes

- Using non-atomic ++ on shared/global bins.
- Assuming shared-memory atomics eliminate contention.
- Omitting initialization or the barrier around private bins.
- Mishandling negative, NaN, exact-upper-bound, or out-of-range values.
- Using counters too narrow for n.
- Ignoring skew and only testing uniform data.

## 10. Edge Cases / Special Cases

Define behavior for empty input, zero bins, invalid categories, NaNs, and values exactly on boundaries. All values in one bin is the contention worst case. Very many bins may make dense privatization impossible. Weighted negative values make the result an aggregate rather than a count. Multi-GPU histograms require a final cross-device reduction.

## 11. How to Explain in Interview

“A GPU histogram is a keyed reduction. Threads classify inputs into bins, but concurrent increments collide, so atomics are needed. To reduce contention, I build per-block or per-warp private histograms in shared memory and merge them globally. The design depends on bin count, input skew, counter width, and boundary policy.”

## 12. Quick Revision Notes

- Histogram = classify + count; output has B bins.
- Race-free update needs atomics or exclusive ownership.
- Privatize locally, merge globally.
- Skew, bin count, and shared-memory capacity drive performance.
- Trap: exact maximum can map one past the last bin.

## 13. Practice Tasks

1. Implement global-atomic and block-private 256-bin histograms.
2. Test uniform, one-hot, and Zipf-like distributions.
3. Add weighted values and 64-bit counters.
4. Convert the histogram to a CDF with scan.
5. Profile atomic throughput and shared-memory occupancy.

## 14. Final Cheat Sheet

| Item       | Must remember                                                      |
|------------|--------------------------------------------------------------------|
| Core       | Parallel classification followed by collision-safe bin aggregation |
| Main issue | Contention, not input parallelism                                  |

| Item            | Must remember                                                             |
|-----------------|---------------------------------------------------------------------------|
| Standard fix    | Per-block/warp privatization then merge                                   |
| Most asked      | Atomics, skew, bin mapping, counter width                                 |
| One-line answer | "A histogram is a keyed reduction optimized by privatizing hot counters." |

## Scatter/Gather

### 1. Overview

**Definition.** Gather reads from data-dependent source indices: `out[i] = in[index[i]]`. Scatter writes to data-dependent destinations: `out[index[i]] = in[i]`. They are the inverse movement patterns when indices form a permutation, but duplicates or missing indices break that simple inverse.

These primitives appear in embeddings, sparse matrices, graph processing, sorting, mesh updates, databases, and data layout conversion. Interviewers ask about coalescing, cache behavior, bounds, duplicate destinations, atomics, determinism, and push-versus-pull formulations.

### 2. Core Idea

Think of gather as each worker receiving a shopping list and fetching one shelf item. Each worker owns one output, so writes are naturally race-free; reads may be scattered. Scatter gives each worker a parcel and destination address. Reads are regular, but two parcels may target the same destination.

```
// gather
if (i < n) out[i] = in[index[i]];

// scatter-add: duplicates are legal
if (i < n) atomicAdd(&out[index[i]], in[i]);
```

Example with `in=[A,B,C,D]`, `index=[2,0,3,1]`: gather yields `[C,A,D,B]`. Scatter yields `[B,D,A,C]`. If scatter indices are `[1,1,...]`, plain assignment races; the required semantic may be last-writer, reject duplicates, or combine with sum/min/max.

### 3. Important Subtopics

| Subtopic            | Meaning / why it matters                                                               | Example                       | Interview angle                     |
|---------------------|----------------------------------------------------------------------------------------|-------------------------------|-------------------------------------|
| Indirect addressing | An index array determines addresses.                                                   | Embedding lookup by token ID. | Bounds and index width.             |
| Coalescing          | Adjacent lanes should touch nearby addresses; random indices create many transactions. | Sort indices before access.   | Useful work versus reordering cost. |

| Subtopic            | Meaning / why it matters                                    | Example                                      | Interview angle                          |
|---------------------|-------------------------------------------------------------|----------------------------------------------|------------------------------------------|
| Scatter conflicts   | Duplicate destinations create write-write hazards.          | Graph edges update same vertex.              | Atomic combine or ownership.             |
| Permutation         | Every destination occurs exactly once.                      | Array reorder.                               | Scatter is race-free and invertible.     |
| Scatter-reduce      | Conflicts intentionally combine associatively.              | <code>out[j] += value.</code>                | Atomics versus sort/segmented reduction. |
| Push vs pull        | Push scatters updates; pull gathers contributors.           | BFS frontier expansion vs vertex inspection. | Races versus extra reads.                |
| Locality reordering | Reorder indices/items to improve cache/coalescing.          | Bucket by destination.                       | Must preserve required order/stability.  |
| Vector payload      | Moving structures may cause alignment and bandwidth issues. | Gather <code>(x,y,z)</code> fields.          | AoS versus SoA.                          |

## 4. Real-World Example

An embedding layer gathers one vector row per token ID. IDs can be random, so row accesses are not globally coalesced, but vector elements within each row can be assigned to adjacent lanes for coalesced loads. Backpropagation scatters gradient vectors back to rows; repeated token IDs require atomic addition or sorting IDs followed by segmented reduction.

## 5. Diagrams / Mental Models

Gather (output-owned)

index: [2,0,3,1]

out[0] ← in[2]

Gather risk: irregular reads

Scatter (input-owned)

index: [2,0,3,1]

in[0] → out[2]

in[1] → out[0]

Scatter risk: conflicting writes

| Pattern | Regular side  | Irregular side | Race risk    |
|---------|---------------|----------------|--------------|
| Gather  | output writes | input reads    | usually none |
| Scatter | input reads   | output writes  | duplicates   |

## 6. Common Interview Questions

| Question                  | Answer                                                    | Expected   | Common mistake  |
|---------------------------|-----------------------------------------------------------|------------|-----------------|
| 1. Define gather/scatter. | Gather uses indirect reads; scatter uses indirect writes. | Equations. | Reversing them. |

| Question                              | Answer                                                                                                         | Expected                       | Common mistake                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------|-------------------------------------------|
| 2. Which is easier to make race-free? | Gather, because one thread normally owns each output.                                                          | Ownership.                     | Saying reads can race destructively.      |
| 3. What if scatter indices repeat?    | Define semantics; use atomics, sort/reduce, or exclusive destination ownership.                                | Conflict policy.               | Plain non-atomic assignment.              |
| 4. Why can gather be slow?            | Warp lanes may request unrelated cache lines, wasting transactions and locality.                               | Memory coalescing.             | Blaming arithmetic.                       |
| 5. When are they inverses?            | When indices are a bijective permutation and inverse indices are used.                                         | Duplicates/holes caveat.       | Assuming any index list is invertible.    |
| 6. Push vs pull?                      | Push scatters from active sources; pull gathers at destinations.                                               | Race/work tradeoff.            | Treating them as identical cost.          |
| 7. How validate indices?              | At a trust boundary, check <code>0 &lt;= index[i] &lt; size</code> ; otherwise guarantee through construction. | Memory safety.                 | Omitting negative signed indices.         |
| 8. Atomics or sort-reduce?            | Atomics suit low contention/modest work; sort-reduce suits heavy conflicts or when grouping is reusable.       | Distribution-dependent choice. | Absolute rule.                            |
| 9. How improve locality?              | Reorder/group indices, process tiles, cache reused values, or change data layout.                              | Include reorder overhead.      | Sorting when one pass cannot amortize it. |
| 10. How handle large payloads?        | Move indices, use structure-of-arrays, and assign lanes cooperatively across payload elements.                 | Bandwidth/layout.              | One thread copying a huge record.         |

## 7. Deep-Dive Questions

- How make scatter deterministic?** Give each destination a unique owner or sort contributions by destination and source, then reduce in a fixed order. Atomic floating-point addition is generally order-dependent.
- How choose push or pull for BFS?** Push is efficient for a small frontier but suffers collisions as it grows; pull examines unvisited vertices and can stop on the first frontier neighbor, often winning for dense frontiers.
- Can cache help random gather?** Yes when indices repeat or fit working sets; read-only/texture paths may help, but they cannot restore coalescing for uniformly random large data.
- How avoid atomics for a permutation scatter?** Prove or construct one-to-one indices; every thread then owns a distinct address.
- What security issue exists?** Untrusted indices can cause out-of-bounds device access; validate before dereference and use overflow-safe address calculations.

## 8. Comparison Tables

| Aspect             | Gather              | Scatter                 | Scatter-reduce                 |
|--------------------|---------------------|-------------------------|--------------------------------|
| Formula            | $o[i] = x[idx[i]]$  | $o[idx[i]] = x[i]$      | $o[idx[i]] \text{ op } x[i]$   |
| Natural owner      | output              | input                   | input contribution             |
| Duplicate index    | repeated read, safe | ambiguous/racy          | combined atomically or grouped |
| Typical bottleneck | read locality       | write locality/conflict | contention                     |

## 9. Common Mistakes

- Reversing gather and scatter definitions.
- Assuming duplicate scatter destinations do not occur.
- Using atomics without defining the combine semantics.
- Ignoring invalid indices and integer overflow.
- Calling allocation contiguous and therefore accesses coalesced.
- Reordering indices while accidentally changing required output order.

## 10. Edge Cases / Special Cases

Handle empty arrays, repeated indices, missing destinations, negative/out-of-range indices, self-aliasing, and overlapping input/output. A permutation can scatter without atomics. Scatter assignment needs a deterministic winner if duplicates are legal. Scatter-add must define overflow/NaN behavior. Index sorting may be profitable only when reused across several operations.

## 11. How to Explain in Interview

"Gather gives each output thread an indirect source, so it is normally race-free but can have uncoalesced reads. Scatter gives each input thread an indirect destination, so duplicate indices can race; I need a permutation guarantee, atomics, or grouping and reduction. I choose push/scatter versus pull/gather by frontier density, contention, and locality."

## 12. Quick Revision Notes

- Gather: indirect read; scatter: indirect write.
- Gather risks poor locality; scatter risks collisions.
- Duplicate scatter needs an explicit semantic.
- Permutation scatter is conflict-free.
- Trap: contiguous arrays do not imply coalesced indirect accesses.

## 13. Practice Tasks

1. Implement bounds-checked gather and permutation scatter.
2. Add scatter-sum with duplicates and compare atomics against sort/reduce.
3. Generate sequential, random, and Zipf indices; measure bandwidth.
4. Implement an embedding gather with lanes cooperating over vector width.
5. Explain a graph algorithm in both push and pull forms.

## 14. Final Cheat Sheet

| Item            | Must remember                                                                      |
|-----------------|------------------------------------------------------------------------------------|
| Gather          | <code>out[i]=in[index[i]]</code> ; irregular reads                                 |
| Scatter         | <code>out[index[i]]=in[i]</code> ; irregular/conflicting writes                    |
| Main decision   | ownership, locality, duplicate policy                                              |
| Most asked      | races, coalescing, atomics, push vs pull                                           |
| One-line answer | "Gather owns outputs; scatter owns inputs and must resolve destination conflicts." |

## Map/Filter

### 1. Overview

**Definition.** Map applies a function independently to every element:  $y[i]=f(x[i])$ . Filter keeps only elements satisfying a predicate while preserving or not preserving their order. Map is naturally one-to-one; filter is variable-output and is implemented as predicate, scan, and scatter.

They underpin image transforms, ETL, query engines, tensor operations, stream processing, and preprocessing. Interviewers ask about data parallelism, fusion, compaction, divergence, output sizing, stability, and memory bandwidth.

### 2. Core Idea

Map is an assembly line where each worker transforms one item with no communication. For  $[1,2,3]$  and  $f(x)=x^2$ , output is  $[1,4,9]$ .

Filter first turns decisions into flags, scans flags into unique positions, then scatters kept items:

```
x:          [7, 2, 9, 4]
x even?: [0, 1, 0, 1]
scan:     [0, 0, 1, 1] (exclusive)
write:      2→0    4→1
output:   [2, 4]
```

```
flag[i] = predicate(x[i]);
pos = exclusive_scan(flag);
if (flag[i]) out[pos[i]] = transform(x[i]);
```

The transform can be fused into the final write, avoiding an intermediate mapped array.

### 3. Important Subtopics

| Subtopic          | Meaning / why it matters                                            | Example                       | Interview angle                          |
|-------------------|---------------------------------------------------------------------|-------------------------------|------------------------------------------|
| Pure map          | Each output depends only on its input; no synchronization.          | Celsius to Fahrenheit.        | Embarrassingly parallel.                 |
| Stream compaction | Dense output from sparse keep flags.                                | Remove invalid records.       | Predicate + exclusive scan + scatter.    |
| Stable filter     | Kept items preserve input order.                                    | Scan-based positions.         | Atomics may not preserve order.          |
| Unstable filter   | Any output order is acceptable.                                     | Per-block/atomic reservation. | Faster alternatives.                     |
| Fusion            | Combine adjacent maps, predicate, or output transform.              | Parse + validate + convert.   | Reduce memory passes/launches.           |
| Divergence        | Different predicate/control paths within a warp reduce utilization. | Complex string rule.          | Predication versus branch cost.          |
| Selectivity       | Fraction kept changes output traffic and best implementation.       | 1% versus 99%.                | Scan still reads all flags.              |
| Output sizing     | Exact count is known only after aggregation/scan.                   | Last offset + last flag.      | Two-phase API or upper-bound allocation. |

## 4. Real-World Example

A database executes `SELECT price*tax FROM orders WHERE status='PAID'`. One kernel evaluates status flags; scan assigns compact positions; kept rows write transformed prices. If the result buffer can be sized to input length, a fused implementation avoids materializing flags/positions through library compaction or block-local buffering.

## 5. Diagrams / Mental Models

```

Map:      n inputs — independent f —> n outputs
Filter: n inputs — predicate —> flags — scan —> positions — scatter —> m
         outputs
Map+filter fusion: evaluate once, write transformed kept values

```

| Property          | Map                  | Filter                     |
|-------------------|----------------------|----------------------------|
| Cardinality       | exactly $n$          | $0..n$                     |
| Communication     | none                 | scan/allocation            |
| Stable by default | index-preserving     | requires ordered positions |
| Main cost         | function + bandwidth | predicate + scan + scatter |

## 6. Common Interview Questions

| Question                        | Answer                                                                                                  | Expected                       | Common mistake                                    |
|---------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------|---------------------------------------------------|
| 1. Why is map easy on GPU?      | Elements are independent and have one output each.                                                      | One thread/item, bounds guard. | Adding unnecessary synchronization.               |
| 2. How implement stable filter? | Predicate to flags, exclusive scan to positions, conditional scatter.                                   | Full pipeline.                 | Atomic counter while claiming stability.          |
| 3. How get filtered size?       | Total flags or <code>pos[n-1]+flag[n-1]</code> .                                                        | Exclusive-scan detail.         | Using last position alone.                        |
| 4. What is compaction?          | Removing rejected/invalid entries into a dense array.                                                   | Filter synonym/pattern.        | Confusing with compression.                       |
| 5. Why fuse maps?               | Avoid intermediate global reads/writes and launch overhead.                                             | Memory traffic.                | Fusing until register pressure becomes harmful.   |
| 6. Stable vs unstable filter?   | Stable preserves relative order; unstable may reserve output slots in arbitrary order.                  | Semantic/performance tradeoff. | Assuming filter never promises order.             |
| 7. How does selectivity matter? | It controls scatter traffic and can justify specialized sparse/dense paths.                             | Still evaluate predicate.      | Assuming 0% kept means zero work.                 |
| 8. Can map run in place?        | Yes if each output only needs the same input element and representations safely overlap.                | No cross-element dependency.   | Generalizing to neighbors.                        |
| 9. What causes divergence?      | Lanes taking different data-dependent branches execute paths with inactive lanes.                       | SIMT behavior.                 | Saying branch always serializes the entire GPU.   |
| 10. When use atomic filtering?  | When order is irrelevant and contention/output size are acceptable, often with block-level reservation. | Simpler unstable compaction.   | One global atomic per kept item under heavy load. |

## 7. Deep-Dive Questions

- How reduce scan overhead?** Warp ballot/popcount gives local ranks, one atomic reserves a block/warp output range, then lanes write within it; global order is generally not stable.
- How decide fusion boundaries?** Fuse when it removes materialized intermediates, but stop if register pressure, code size, repeated computation, or scheduling reduces throughput.
- How filter variable-sized outputs?** Map each item to a size, scan sizes into byte/element offsets, allocate the total, then generate into disjoint ranges.
- How preserve stability across blocks?** Use a global scan of block/item counts so every block receives an offset ordered before later blocks.

5. **What if predicate is expensive?** Store flags/results if recomputation costs more than memory traffic; otherwise recompute to save storage. Profile the tradeoff.

## 8. Comparison Tables

| Strategy              | Stable                         | Extra data                             | Strength                | Weakness                        |
|-----------------------|--------------------------------|----------------------------------------|-------------------------|---------------------------------|
| Flag + scan + scatter | yes                            | flags/positions or fused library state | deterministic, scalable | multiple phases                 |
| Global atomic append  | no                             | counter                                | simple                  | contention/order nondeterminism |
| Block reservation     | block order usually not global | block counts/buffers                   | fewer atomics           | more logic                      |

## 9. Common Mistakes

- Treating filter as independent writes without assigning unique positions.
- Off-by-one output count with exclusive scan.
- Claiming atomic append is stable.
- Materializing every intermediate map unnecessarily.
- Ignoring branch divergence, register pressure, and transfer overhead.
- Performing in-place stable compaction without protecting unread input.

## 10. Edge Cases / Special Cases

Empty input, keep-none, and keep-all must produce sizes  $0$ ,  $0$ , and  $n$ . An in-place filter can overwrite elements another thread has not read. Predicates involving NaN need explicit semantics. Variable-size outputs need overflow-safe size scans. Side-effecting map functions are not freely reorderable and undermine the map model.

## 11. How to Explain in Interview

"Map applies an independent function per element, so it maps directly to GPU threads. Filter has variable output positions, so a stable implementation computes predicate flags, exclusive-scans them into unique offsets, and scatters kept items. I fuse adjacent transforms when that removes memory passes, while watching divergence and register pressure."

## 12. Quick Revision Notes

- Map: one-to-one, independent, usually bandwidth-bound.
- Filter: predicate  $\rightarrow$  scan  $\rightarrow$  scatter.
- Stable filter preserves order; atomic append usually does not.
- Exact output count comes from total flags.
- Trap: in-place compaction can overwrite unread data.

## 13. Practice Tasks

1. Implement square-map and stable even-number filter.
2. Fuse transform with filter output and compare memory traffic.
3. Implement unstable block-reservation compaction.
4. Benchmark 0%, 1%, 50%, and 100% selectivity.
5. Extend to variable-length string output using size scan.

## 14. Final Cheat Sheet

| Item            | Must remember                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------|
| Map             | independent $y[i]=f(x[i])$                                                                          |
| Filter          | predicate + scan + conditional scatter                                                              |
| Main comparison | stable scan-based vs unstable atomic append                                                         |
| Most asked      | output size, stability, fusion, divergence                                                          |
| One-line answer | "Map transforms in place by index; filter uses scan to turn keep decisions into compact positions." |

## Sparse Operations

### 1. Overview

**Definition.** Sparse operations store and process only nonzero or structurally present entries. A sparse matrix with `nnz` stored values avoids the `rows × columns` cost of dense storage. Core formats include COO, CSR, CSC, ELL, and hybrids; core kernels include sparse matrix-vector multiply (SpMV), sparse matrix-matrix multiply (SpGEMM), and sparse gather/scatter.

Sparse computation is central to graphs, recommender systems, search, scientific solvers, finite elements, and sparse neural networks. Interviewers ask about format choice, indirect memory access, load imbalance, duplicate entries, arithmetic intensity, and why fewer arithmetic operations do not guarantee speed.

### 2. Core Idea

For dense  $y=Ax$ , every matrix position is visited. CSR stores only values, their column indices, and row boundaries:

```
A = [10 0 2]   values = [10,2,3]
     [ 0 3 0]   col_idx = [ 0,2,1]
                        row_ptr = [ 0,2,3]
```

Row `r` occupies `k=row_ptr[r]..row_ptr[r+1)-1`; SpMV computes  $y[r] += values[k] * x[col\_idx[k]]$ . Parallelism may assign one thread, warp, or block per row. The difficulty is that row lengths vary and `x` reads are indirect.

### 3. Important Subtopics

| Subtopic             | Meaning / why it matters                                                  | Example                                         | Interview angle                                    |
|----------------------|---------------------------------------------------------------------------|-------------------------------------------------|----------------------------------------------------|
| COO                  | Arrays of (row, col, value); simple construction.                         | Edge list.                                      | Duplicates and sorting; extra row indices.         |
| CSR                  | row_ptr, col_idx, values; efficient row traversal.                        | SpMV.                                           | Explain rows+1 pointer array.                      |
| CSC                  | Column-oriented analogue of CSR.                                          | Column access, some factorizations.             | CSR vs CSC based on traversal.                     |
| ELL                  | Fixed slots per row, padded.                                              | Regular stencil matrices.                       | Coalescing versus padding waste.                   |
| SpMV mapping         | Thread/warp/block per row or nonzero.                                     | Warp reduction for a row.                       | Row-length distribution drives choice.             |
| SpGEMM               | Symbolic phase discovers output structure; numeric phase computes values. | $C=A \times B$ .                                | Irregular allocation and duplicates.               |
| Load balance         | Equal rows can have very unequal work.                                    | Power-law graph hubs.                           | Bucket/split long rows or nonzero-based partition. |
| Canonicalization     | Sort indices and combine duplicate coordinates.                           | Two COO entries at (i, j).                      | Required semantics and deterministic results.      |
| Arithmetic intensity | Index and value loads accompany few FLOPs.                                | SpMV $\approx$ multiply-add plus several loads. | Commonly bandwidth/latency bound.                  |

### 4. Real-World Example

PageRank represents the web as a sparse matrix. Each iteration performs a sparse matrix-vector-like update. CSR supports outgoing-row traversal for push; CSC supports incoming-neighbor gathering for pull. Power-law degree distributions require special handling for hubs, and repeatedly converting formats would erase the benefit, so the representation follows the dominant traversal.

### 5. Diagrams / Mental Models

```

row_ptr: [0, 2, 3, 6]
         |--r0-| |-r1| |---r2---|
values:  [a  b   c   d  e  f]
cols:    [0  4   2   0  1  4]

row length = row_ptr[r+1] - row_ptr[r]
```

| Format | Best access              | Storage overhead                 | Weakness              |
|--------|--------------------------|----------------------------------|-----------------------|
| COO    | construction/edge stream | row+col per entry                | weak row lookup       |
| CSR    | rows                     | one col per value + row pointers | irregular row lengths |
| CSC    | columns                  | one row per value + col pointers | weak row traversal    |
| ELL    | regular rows             | padding                          | waste for skew        |

## 6. Common Interview Questions

| Question                                 | Answer                                                                                          | Expected                                 | Common mistake                       |
|------------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------|
| 1. What does <b>nnz</b> mean?            | Number of explicitly stored entries, usually nonzeros.                                          | Structural versus numerical zero nuance. | Using matrix size.                   |
| 2. Explain CSR.                          | Row pointers delimit contiguous value/column slices; <b>row_ptr</b> has <b>rows+1</b> entries.  | Example and final pointer= <b>nnz</b> .  | Storing a pointer per nonzero.       |
| 3. CSR vs COO?                           | CSR traverses rows efficiently; COO is easier to build/reorder.                                 | Conversion cost and duplicates.          | Calling one universally better.      |
| 4. Why is SpMV often bandwidth-bound?    | Each nonzero does little arithmetic while loading value, index, and irregular vector data.      | Low arithmetic intensity.                | Focusing on peak FLOP/s.             |
| 5. Thread or warp per row?               | Threads suit short uniform rows; warps cooperate on longer rows and reduce partials.            | Distribution-dependent policy.           | Fixed answer without row statistics. |
| 6. What causes load imbalance?           | Unequal row lengths and irregular work/contention.                                              | Power-law examples.                      | Equating row count with work.        |
| 7. Why can dense be faster?              | Dense kernels have regular accesses and high reuse; moderate density can outweigh wasted zeros. | Crossover depends on hardware/shape.     | Assuming sparse always wins.         |
| 8. What are duplicate entries?           | Multiple stored coordinates for one logical location; combine by specified operator.            | Canonicalization.                        | Silently taking the last value.      |
| 9. Why separate symbolic/numeric SpGEMM? | Output positions/counts must be discovered before values can be allocated/computed.             | Irregular output size.                   | Assuming output <b>nnz</b> is known. |
| 10. How improve locality?                | Reorder rows/columns, cache <b>x</b> , use suitable format, and group similar                   | Preprocessing amortization.              | Reordering while losing              |

| Question | Answer       | Expected | Common mistake |
|----------|--------------|----------|----------------|
|          | row lengths. |          | external IDs.  |

## 7. Deep-Dive Questions

1. **How parallelize one long CSR row?** Split its nonzeros across a warp/block, multiply independently, then reduce partial sums. Very long rows may be split across blocks with an additional reduction/atomic.
2. **What is segmented reduction in sparse kernels?** Nonzeros are processed as a stream; row boundaries mark segments, and products within each row segment reduce to one output.
3. **How choose a format automatically?** Measure dimensions, density, row-length variance, access direction, reuse count, and conversion cost; hybrid formats may isolate regular and exceptional rows.
4. **How handle dynamic sparsity?** COO/hash-like structures ease insertion; periodically sort, combine, and compress to CSR for repeated compute. Dynamic CSR insertion is expensive.
5. **Why might reordering help?** It can improve vector/cache locality and reduce bandwidth or warp imbalance, but preprocessing and mapping results back must be amortized.

## 8. Comparison Tables

| Aspect     | Dense GEMV                          | CSR SpMV                        |
|------------|-------------------------------------|---------------------------------|
| Work       | $O(\text{rows} \times \text{cols})$ | $O(\text{nnz})$                 |
| Access     | regular                             | values regular, vector indirect |
| Metadata   | none per value                      | column index + row pointers     |
| Best case  | dense/high reuse                    | very low density                |
| Main limit | bandwidth                           | bandwidth + latency/imbalance   |

## 9. Common Mistakes

- Assuming sparse storage is faster merely because it stores fewer values.
- Confusing `row_ptr` offsets with column indices.
- Ignoring sortedness and duplicate-coordinate semantics.
- Assigning one thread per row despite extreme row-length skew.
- Forgetting index traffic, conversion cost, and 32/64-bit limits.
- Counting explicit zeros as automatically absent.

## 10. Edge Cases / Special Cases

Empty rows have equal adjacent row pointers. `nnz=0` must produce zero output. Explicit zeros may remain stored. Duplicate coordinates require combination. Extremely long rows need cooperative splitting. Invalid or unsorted indices can break kernels that assume canonical CSR. Rectangular matrices and transpose operations change the preferred layout. Index arithmetic may overflow 32 bits.

## 11. How to Explain in Interview

"Sparse operations process only stored entries, usually  $O(nnz)$ , but pay for index metadata, indirect accesses, and load imbalance. In CSR, row pointers delimit each row's values and column indices. For SpMV I choose thread-, warp-, or block-per-row based on row lengths, and I treat format conversion, duplicates, locality, and bandwidth as first-class costs."

## 12. Quick Revision Notes

- CSR arrays: `row_ptr[rows+1]`, `col_idx[nnz]`, `values[nnz]`.
- COO builds easily; CSR traverses rows; CSC traverses columns; ELL favors regular rows.
- SpMV is usually memory-bound and irregular.
- `row_ptr[r+1] - row_ptr[r]` is row length.
- Trap: sparse is not automatically faster than dense.

## 13. Practice Tasks

1. Convert a small matrix manually among COO, CSR, and CSC.
2. Implement CPU CSR SpMV, then GPU thread-per-row and warp-per-row versions.
3. Test empty rows, duplicates, unsorted columns, and one huge row.
4. Plot runtime against density and row-length variance.
5. Build CSR from COO using histogram, scan, and scatter.

## 14. Final Cheat Sheet

| Item             | Must remember                                                                                |
|------------------|----------------------------------------------------------------------------------------------|
| Core             | Store/process <code>nnz</code> structural entries plus indices                               |
| Main format      | CSR for rows, CSC for columns, COO for construction                                          |
| Main bottlenecks | metadata, indirect memory, imbalance                                                         |
| Most asked       | CSR layout, SpMV mapping, sparse vs dense, duplicates                                        |
| One-line answer  | "Sparse kernels save zero work but trade regular compute for metadata and irregular memory." |

## Parallel Graph Traversal

### 1. Overview

**Definition.** Parallel graph traversal explores vertices and edges concurrently. Breadth-first search (BFS) is the canonical GPU traversal: it visits vertices by distance level using a frontier. Related traversals drive connected components, shortest paths, reachability, PageRank, and graph analytics.

Graphs matter in social networks, routing, fraud detection, compilers, databases, and recommendation systems. Interviewers ask because graphs expose irregular memory, unpredictable frontier sizes, duplicate discovery, synchronization, load imbalance, representation choice, and push-versus-pull execution.

## 2. Core Idea

BFS maintains a current frontier containing vertices at distance  $d$ . Threads inspect frontier edges. An undiscovered neighbor is atomically claimed, assigned distance  $d+1$ , and appended to the next frontier. A kernel boundary (or controlled device-wide mechanism) separates levels.

```
Graph: A → {B,C}, B → {D}, C → {D,E}
level 0 frontier: [A]
level 1 frontier: [B,C]
level 2 frontier: [D,E] (D is claimed once despite two parents)
```

```
for each u in frontier in parallel:
  for each v in neighbors(u):
    if atomicCAS(&distance[v], INF, distance[u] + 1) == INF:
      pos = atomicAdd(next_count, 1);
      next_frontier[pos] = v;
```

Correct atomic claim prevents two threads from both treating  $v$  as newly discovered. Efficient implementations reduce append contention with warp/block queues or generate flags and compact them.

## 3. Important Subtopics

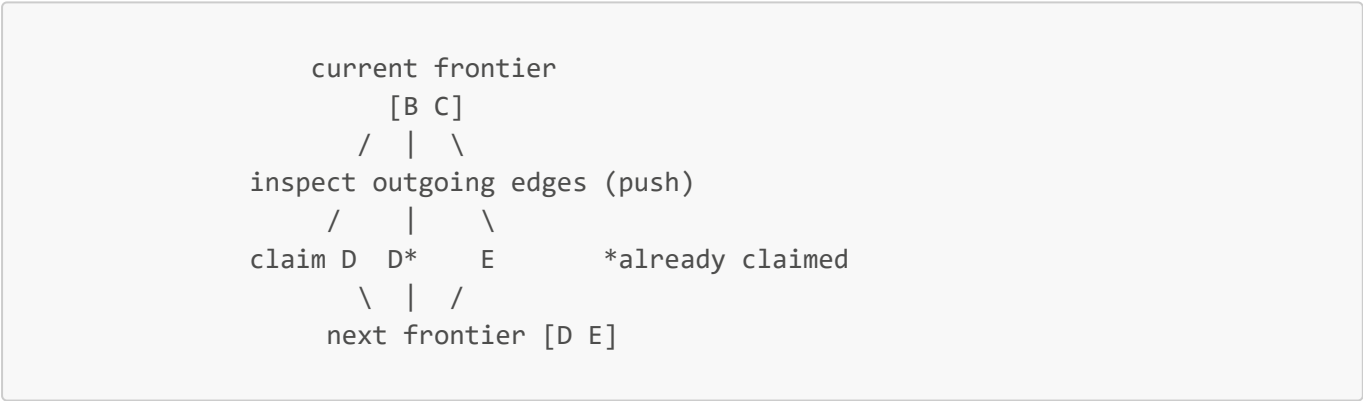
| Subtopic         | Meaning / why it matters                                        | Example                                 | Interview angle                                  |
|------------------|-----------------------------------------------------------------|-----------------------------------------|--------------------------------------------------|
| Frontier         | Active vertices for the current level/iteration.                | BFS queue by level.                     | Sparse list versus dense bitmap.                 |
| CSR adjacency    | Neighbors of a vertex are contiguous.                           | <code>row_ptr[u]..row_ptr[u+1]</code> . | Good edge streaming, irregular degrees.          |
| Vertex-centric   | One worker/group per active vertex.                             | Expand each frontier vertex.            | Hub imbalance.                                   |
| Edge-centric     | Workers process edges, often with source activity checks.       | Balanced edge work.                     | Extra scanning/filtering.                        |
| Push/top-down    | Active vertices scatter to neighbors.                           | Small frontier.                         | Duplicate writes and atomics.                    |
| Pull/bottom-up   | Unvisited vertices gather/check whether any neighbor is active. | Dense frontier.                         | More vertices read, fewer write conflicts.       |
| Atomic discovery | Compare-and-swap changes unvisited to visited once.             | <code>CAS(INF, d+1)</code> .            | Atomic append alone does not prevent duplicates. |

| Subtopic            | Meaning / why it matters                                   | Example                        | Interview angle                        |
|---------------------|------------------------------------------------------------|--------------------------------|----------------------------------------|
| Load balance        | Degree varies widely.                                      | One hub has millions of edges. | Split/virtualize high-degree vertices. |
| Frontier compaction | Convert discovered flags/items into a dense next frontier. | Scan or block queue.           | Reuse filter primitive.                |
| Termination         | Stop when next frontier is empty or convergence occurs.    | BFS complete.                  | Device-host synchronization cost.      |

4. Real-World Example

A fraud service explores transactions up to three hops from a suspicious account. The graph is stored in CSR on the GPU. Early BFS levels use push because the frontier is small. When the frontier covers a large fraction of vertices, direction-optimizing BFS switches to pull over unvisited accounts. Visited state prevents cycles, and parent IDs reconstruct an evidence path.

5. Diagrams / Mental Models



| Phase               | Work                   | Synchronization          |
|---------------------|------------------------|--------------------------|
| Expand frontier     | inspect incident edges | atomic discovery         |
| Build next frontier | append/compact winners | atomic/scan/block queues |
| Advance level       | swap frontier buffers  | kernel or grid boundary  |
| Finish              | next size is zero      | counter/status check     |

6. Common Interview Questions

| Question                                 | Answer                                                                                                                 | Expected                             | Common mistake                                             |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------|
| 1. How parallelize BFS?                  | Process current-frontier vertices/edges concurrently, claim unseen neighbors, build next frontier, repeat levels.      | Level ordering.                      | Launching all levels together without coordination.        |
| 2. Why atomically claim visited?         | Several parents can discover one vertex simultaneously; only one should enqueue it.                                    | CAS winner semantics.                | Atomic counter only, allowing duplicates.                  |
| 3. Why is BFS difficult on GPU?          | Irregular degree, random accesses, dynamic frontier, divergence, and contention.                                       | Hardware-aware explanation.          | Saying BFS has insufficient parallelism universally.       |
| 4. Push vs pull?                         | Push scans edges from active vertices; pull scans unvisited vertices for active incoming neighbors.                    | Frontier-density tradeoff.           | Always preferring one direction.                           |
| 5. Why CSR?                              | It compactly stores adjacency and streams a vertex's neighbors contiguously.                                           | $O(V+E)$ storage.                    | Claiming neighbor accesses to state become contiguous too. |
| 6. How handle hubs?                      | Assign warps/blocks or split edge ranges rather than one thread per hub.                                               | Load balancing.                      | Same mapping for every degree.                             |
| 7. Complexity of BFS?                    | $O(V+E)$ work in standard adjacency traversal; parallel depth is related to graph diameter/levels plus per-level work. | Work not simply $O(\text{levels})$ . | Ignoring repeated pull scans/implementation overhead.      |
| 8. How avoid frontier append bottleneck? | Aggregate locally, reserve chunks per warp/block, or compact flags with scan.                                          | Hierarchical allocation.             | One hot atomic per candidate.                              |
| 9. BFS vs DFS on GPU?                    | BFS exposes a broad frontier; DFS has a sequential stack/deep dependency and is usually less GPU-friendly.             | Workload shape.                      | Saying DFS cannot be parallelized.                         |
| 10. How recover shortest path?           | First successful discoverer stores parent; follow parent links from target.                                            | BFS unweighted shortest path.        | Overwriting parent on later discoveries.                   |

## 7. Deep-Dive Questions

1. **When switch direction?** Estimate push work as outgoing edges from the frontier and pull work as edges inspected from unvisited vertices; switch when the frontier/edge volume becomes dense, with hysteresis to avoid oscillation.

2. **How process weighted graphs?** Ordinary BFS is insufficient. Use delta-stepping, frontier-based Bellman–Ford, or another shortest-path algorithm; relax distances atomically and tolerate/reduce duplicate work.
3. **How represent the frontier?** Lists are efficient when sparse; bitmaps offer constant membership checks and suit pull when dense. Hybrid implementations convert based on frontier density.
4. **How make parent choice deterministic?** Use an ordered tie rule, such as atomic minimum parent after establishing the level, rather than accepting whichever thread wins scheduling.
5. **How traverse graphs larger than one GPU?** Partition vertices/edges, perform local expansion, exchange remote frontier updates, deduplicate, and synchronize levels; partition quality and communication dominate.

## 8. Comparison Tables

| Aspect        | Push BFS                  | Pull BFS                            |
|---------------|---------------------------|-------------------------------------|
| Iterates over | active sources            | unvisited destinations              |
| Frontier best | small/sparse              | large/dense                         |
| Access        | outgoing adjacency        | incoming adjacency                  |
| Main cost     | collisions/atomic updates | scanning inactive possibilities     |
| Early exit    | no per source             | yes after finding frontier neighbor |

| Aspect                   | BFS                   | DFS                     |
|--------------------------|-----------------------|-------------------------|
| Structure                | frontier/queue        | stack/recursion         |
| Unweighted shortest path | yes                   | no                      |
| GPU parallelism          | often broad per level | irregular/deep          |
| Synchronization          | between levels        | dynamic task scheduling |

## 9. Common Mistakes

- Marking visited after enqueue, allowing duplicate frontier entries.
- Using a non-atomic check-then-set for discovery.
- Assuming one thread per vertex balances power-law graphs.
- Forgetting that pull needs efficient incoming adjacency or a transpose.
- Treating kernel-level barriers as grid-wide.
- Claiming BFS depth is  $O(\log V)$  for arbitrary graphs; a path has  $V-1$  levels.
- Measuring only edge processing while ignoring frontier construction and synchronization.

## 10. Edge Cases / Special Cases

Disconnected vertices remain unreachable. Self-loops and parallel edges must not duplicate discoveries. An empty source set terminates immediately; invalid sources must be rejected. A path graph offers little frontier parallelism, while a star graph creates a sudden huge frontier/hub. Directed graphs distinguish incoming and outgoing adjacency. Distance width must cover the maximum path. Dynamic graphs require consistent snapshots or defined concurrent-update semantics.

## 11. How to Explain in Interview

"GPU BFS proceeds level by level with a frontier. Threads expand active vertices through CSR adjacency, atomically claim undiscovered neighbors, and compact winners into the next frontier. The hard parts are irregular memory, degree imbalance, duplicate discovery, and frontier construction. I use push for sparse frontiers, pull for dense ones, and adapt thread/warp/block ownership to vertex degree."

## 12. Quick Revision Notes

- BFS state: distance/visited + current frontier + next frontier.
- Atomic claim must precede enqueue.
- CSR gives contiguous neighbor lists, not necessarily contiguous vertex-state reads.
- Push small frontiers; pull dense frontiers.
- $O(V+E)$  standard work; parallel levels depend on graph diameter.
- Trap: block barrier cannot separate BFS levels device-wide.

## 13. Practice Tasks

1. Trace frontier, distance, and parent arrays on a cyclic graph.
2. Implement CPU frontier BFS, then a CUDA-like push version with atomic claim.
3. Replace global per-item append with block queues or scan compaction.
4. Implement pull BFS using a frontier bitmap and compare switching thresholds.
5. Benchmark path, grid, star, random, and power-law graphs.
6. Extend traversal to connected components or unweighted shortest-path recovery.

## 14. Final Cheat Sheet

| Item             | Must remember                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------|
| Core             | Level-synchronous frontier expansion over graph edges                                                    |
| Correctness      | Atomic first discovery, then enqueue                                                                     |
| Representation   | CSR adjacency; list/bitmap frontier                                                                      |
| Main bottlenecks | irregular memory, degree skew, collisions, per-level sync                                                |
| Most asked       | push vs pull, atomics, complexity, load balance                                                          |
| One-line answer  | "Parallel BFS expands a frontier, atomically claims neighbors, and compacts the winners level by level." |

## Cross-Topic Interview Map

| If the problem says...  | Think first about... |
|-------------------------|----------------------|
| "one total/statistic"   | Reduction            |
| "offset for every item" | Prefix sum           |

| If the problem says...                | Think first about...     |
|---------------------------------------|--------------------------|
| "ordered/grouped keys"                | Parallel sorting         |
| "counts per category"                 | Histogram                |
| "indirect move/update"                | Scatter/gather           |
| "transform or keep items"             | Map/filter               |
| "mostly zeros / irregular rows"       | Sparse operations        |
| "neighbors / frontier / reachability" | Parallel graph traversal |

These patterns compose: filter is map + scan + scatter; radix sort is histogram + scan + scatter; CSR construction uses histogram + scan + scatter; graph traversal uses sparse adjacency + gather/scatter + filter/compaction; and each stage often ends with a reduction for counts or convergence.